```
#include <iostream>
#include <vector>

using namespace std;


/*
Required implementation:
- Implement a stack using an array-based representation.
    **DONE
- Implement a stack using your own linked-list representation.
    **DONE
- Implement a queue using an array-based representation.
    **DONE

- Implement a queue using your own linked-list representation.

- Implement a deque using an array-based representation.
    **DONE

- Implement a deque using your own linked-list representation.

- For each structure, demonstrate insertion and removal operations, empty handling, and
  a sequence of at least 10 operations.

- Explain which operations are O(1) and which can become O(n) in your particular design.

- Restrictions: complete the assignment in C++17. Do not use <stack>, <queue>, <deque>,
  or another container that directly implements the required ADT.
*/

//Setting up Node for my linked list
class ArrayStack{
private:
    int allocationSize;
    int* array;
    int length;
    int maxLength;

    void Resize(){
        int newSize = allocationSize * 2;
        int* newArray = new int[newSize];
        for(int i = 0; i < length; i++){
            newArray[i] = array[i];
        }

        delete[] array;
        array = newArray;
        allocationSize = newSize;
    }

public:

    //################################### ALL OF THESE WILL BE FOR THE IMPLEMENTATION OF ARRAY STACK
###################################
    ArrayStack(int optionalMaxLength = -1){
        allocationSize = 8;
        array = new int[allocationSize];
        length = 0;
        maxLength = optionalMaxLength;
    }

    virtual ~ArrayStack(){
        delete[] array;
```

```
    }

    int GetLength() const {
        return length;
    }

    int getMaxLength() const {
        return maxLength;
    }

    bool Push(int item){
        if (length == maxLength){
            return false;
        }
        else if (length == allocationSize){
            Resize();
        }

        //cout << "Pushing to top of array" << endl;

        array[length] = item;
        length++;
        return true;
    }

    int Pop(){
        if(length == 0){
            cout << "Array stack is empty" << endl;
            return -1;
        }

        int value = array[length - 1];
        array[length - 1] = 0;
        length--;

        //cout << "Popping top of array" << endl;

        return value;
    }

    int peek(){
        if (length == 0){
            cout << "Array stack is empty" << endl;
            return -1;
        }
        return array[length -1];
    }

    void print(){
        for(int i = 0; i < length; i++){
            cout << array[i] << endl;
        }
    }

//#########################################################################################################
####################
};

class ArrayQueue{
private:
    int allocationSize;
    int* array;
    int length;
    int maxLength;
    int frontIndex;
```

```
    void Resize(){
        int newSize = allocationSize * 2;
        int* newArray = new int[newSize];

        for(int i = 0; i < length; i++){
            newArray[i] = array[(frontIndex + i) % allocationSize];
        }

        delete[] array;
        array = newArray;
        allocationSize = newSize;
        frontIndex = 0;
    }

public:
    ArrayQueue(int optionalMaxLength = -1):
        allocationSize(8),
        array(new int[allocationSize]),
        length(0),
        maxLength(optionalMaxLength),
        frontIndex(0){}

    virtual ~ArrayQueue(){
        delete[] array;
    }

    int peek(){
        if (length == 0){
            cout << "Array queue is empty" << endl;
            return -1;
        }
        return array[frontIndex];
    }


    bool Enqueue(int item){
        if (maxLength >= 0 && length >= maxLength){
            return false;
        }
        if (length == allocationSize){
            Resize();
        }

        int itemIndex = (frontIndex + length) % allocationSize;
        array[itemIndex] = item;
        length++;

        return true;
    }

    int Dequeue(){
        if (length == 0){
            cout << "Array queue is empty" << endl;
            return -1;
        }

        int toRetrun = array[frontIndex];
        length--;
        frontIndex = (frontIndex + 1) % allocationSize;
        return toRetrun;
    }

    void print(){
        for(int i = 0; i < length; i++){
```

```
#include <iostream>
#include <vector>

using namespace std;


// Create your node struct method (Kind of like a default where you dont need any arguments to call
on it)
class LinkedStack{
private:
    struct Node{
        int value;
        Node* next;
        Node(int v, Node* n = nullptr) : value(v), next(n) {}
    };

    Node* head;
    int length;
    int maxLength;
public:
    LinkedStack(int optionalMaxLength = -1):
    head(nullptr),
    length(0),
    maxLength(optionalMaxLength){}

    virtual ~LinkedStack(){
        while (head != nullptr){
            Node* next = head->next;
            delete head;
            head = next;
        }
    }

    int GetLength() const{
        return length;
    }

    bool Queue(int item){
        if (maxLength >= 0 && length >= maxLength){
            return false;
        }

        head = new Node{item, head};
        length++;
        return true;
    }

    int Pop(){
        if (head == nullptr){
            cout << "Linked stack is empty" << endl;
            return -1;
        }

        Node* oldHead = head;
        int value = oldHead->value;
        head = head->next;
        delete oldHead;
        length--;
        return value;
    }

    int peek(){
        if (head == nullptr){
            cout << "Linked stack is empty" << endl;
```

```
            return -1;
        }
        cout << "Value peeking is: " ;
        return head->value;
    }

    void print(){
        if(length == 0){
            cout << "Linked stack is empty" << endl;
            return;
        }

        for(Node* cur = head; cur != nullptr; cur = cur->next){
            cout << cur->value << endl;
        }
    }

};

class LinkedQueue{
private:
    struct Node{
        int value;
        Node* next;
        Node(int v, Node* n = nullptr) : value(v), next(n) {}
    };

    Node* head;
    Node* tail;
    int length;
    int maxLength;

public:
    LinkedQueue(int optionalMaxLength = -1):
    head(nullptr),
    tail(nullptr),
    length(0),
    maxLength(optionalMaxLength){}

    virtual ~LinkedQueue(){
        while (head != nullptr){
            Node* next = head->next;
            delete head;
            head = next;
        }
    }

    int GetLength() const{
        return length;
    }

    bool Enqueue(int item){
        if (maxLength >= 0 && length >= maxLength){
            return false;
        }

        Node* node = new Node(item);
        if(tail == nullptr){
            head = tail = node;
        }
        else{
            tail->next = node;
            tail = node;
        }
        length++;
```

```
        return true;
    }

    int Dequeue(){
        if (head == nullptr){
            cout << "Linked stack is empty" << endl;
            return -1;
        }

        Node* oldHead = head;
        int value = oldHead->value;
        head = head->next;
        if(head == nullptr){
            tail = nullptr;
        }
        delete oldHead;
        length--;
        return value;
    }

    int peek(){
        if (head == nullptr){
            cout << "Linked stack is empty" << endl;
            return -1;
        }
        cout << "Peeking the queue: ";
        return head->value;
    }

    void print(){
        for(Node* cur = head; cur != nullptr; cur = cur->next){
            cout << cur->value << endl;
        }
    }

};

class LinkedDeque{
private:
    struct Node{
        int value;
        Node* prev;
        Node* next;
        Node(int v) : value(v), prev(nullptr), next(nullptr) {}
    };

    Node* head;
    Node* tail;
    int length;
    int maxLength;

public:
    LinkedDeque(int optionalMaxLength = -1):
    head(nullptr),
    tail(nullptr),
    length(0),
    maxLength(optionalMaxLength){}

    virtual ~LinkedDeque(){
        while (head != nullptr){
            Node* next = head->next;
            delete head;
            head = next;
        }
    }
```

```
    int GetLength() const{
        return length;
    }

    bool pushFront(int item){
        if (maxLength >= 0 && length >= maxLength){
            return false;
        }

        Node* node = new Node(item);
        if(head == nullptr){
            head = tail = node;
        }
        else{
            node->next = head;
            head->prev = node;
            head = node;
        }
        length++;
        return true;
    }

    bool pushBack(int item){
        if (maxLength >= 0 && length >= maxLength){
            return false;
        }

        Node* node = new Node(item);

        if(tail == nullptr){
            head = tail = node;
        }
        else{
            node->prev = tail;
            tail->next = node;
            tail = node;
        }
        length++;
        return true;
    }

    int popFront(){
        if (head == nullptr){
            cout << "Linked deque is empty" << endl;
            return -1;
        }

        Node* oldhead = head;
        int value = oldhead->value;
        head = head->next;

        if(head == nullptr){
            tail = nullptr;
        }
        else{
            head->prev = nullptr;
        }
        delete oldhead;
        length--;
        return value;

    }

    int popBack(){
```

```
        if (tail == nullptr){
            cout << "Linked deque is empty" << endl;
            return -1;
        }

        Node* oldtail = tail;
        int value = oldtail->value;
        tail = tail->prev;

        if(tail == nullptr){
            head = nullptr;
        }
        else{
            tail->next = nullptr;
        }
        delete oldtail;
        length--;
        return value;
    }

    void print(){
        for(Node* cur = head; cur != nullptr; cur = cur->next){
            cout << cur->value << endl;
        }
    }

};

int main(){

// For linked list
    cout << "Stacking" << endl;
    LinkedStack stack;
    stack.Queue(3);
    stack.Queue(40);
    stack.Pop();
    stack.Queue(1);
    cout<< stack.peek() << endl;
    stack.Queue(56);
    stack.print();
    stack.Pop();
    stack.Pop();
    stack.Queue(1);
    cout << stack.peek() << endl;
    stack.Queue(19);
    stack.Pop();
    stack.print();

    cout << endl;
    cout << "//////////////////////////////" <<  endl;
    cout << endl;

// For linked queue
    cout << "Queueing" << endl;
    LinkedQueue queue;
    queue.Enqueue(1);
    queue.Enqueue(5);
    cout << queue.peek() << endl;
    queue.Enqueue(4);
    queue.Dequeue();
    queue.Dequeue();
    queue.print();
    queue.Enqueue(5);
    queue.Enqueue(23);
    queue.Enqueue(0);
```

```
    queue.Enqueue(67);
    queue.print();

    cout << endl;
    cout << "////////////////////////////////" <<  endl;
    cout << endl;

// For linked deque
    cout << "Dequeing" << endl;
    LinkedDeque deque;
    deque.pushFront(1);
    deque.pushFront(5);
    deque.pushFront(4);
    deque.popBack();
    deque.print();
    deque.pushBack(5);
    deque.pushBack(23);
    deque.pushFront(67);
    deque.print();

    cout << endl;
    cout << "////////////////////////////////" <<  endl;
    cout << endl;




    return 0;
}
```

```
            cout << array[(frontIndex + i) % allocationSize] << endl;
        }
    }


};

class ArrayDeques{
private:
    int allocationSize;
    int* array;
    int length;
    int maxLength;
    int frontIndex;
    int backIndex;

    void Resize(){
        int newSize = allocationSize * 2;
        int* newArray = new int[newSize];

        for(int i = 0; i < length; i++){
            newArray[i] = array[(frontIndex + i) % allocationSize];
        }

        delete[] array;
        array = newArray;
        allocationSize = newSize;
        frontIndex = 0;
        backIndex = length;
    }

public:
    ArrayDeques(int optionalMaxLength = -1):
        allocationSize(8),
        array(new int[allocationSize]),
        length(0),
        maxLength(optionalMaxLength),
        frontIndex(0){}

    virtual ~ArrayDeques(){
        delete[] array;
    }

    bool pushFront(int item){
        if (maxLength >= 0 && length >= maxLength){
            return false;
        }
        if (length == allocationSize){
            Resize();
        }

        frontIndex = (frontIndex + allocationSize -1) % allocationSize;
        array[frontIndex] = item;
        length++;

        return true;
    }

    bool pushBack(int item){
        if (maxLength >= 0 && length >= maxLength){
            return false;
        }
        if (length == allocationSize){
            Resize();
        }
```

```
        array[(frontIndex + length) % allocationSize] = item;
        length++;
        return true;
    }

    int PopFront(){
        if (length == 0){
            cout << "Array queue is empty" << endl;
            return -1;
        }

        int toRetrun = array[frontIndex];
        length--;
        frontIndex = (frontIndex + 1) % allocationSize;
        return toRetrun;
    }

    int PopBack(){
        if (length == 0){
            cout << "Array deque is empty" << endl;
            return -1;
        }
        length--;
        return array[(frontIndex + length) % allocationSize];
    }

    void print(){
        for(int i = 0; i < length; i++){
            cout << array[(frontIndex + i) % allocationSize] << endl;
        }
    }


};

int main(){

// For linked list
    cout << "Stacking" << endl;
    ArrayStack stack;
    stack.Push(3);
    stack.Push(40);
    stack.Pop();
    stack.Push(1);
    cout<< stack.peek() << endl;
    stack.Push(56);
    stack.print();
    stack.Pop();
    stack.Pop();
    stack.Push(1);
    cout << stack.peek() << endl;
    stack.Push(19);
    stack.Pop();
    stack.print();

    cout << endl;
    cout << "////////////////////////////" <<  endl;
    cout << endl;

// For linked queue
    cout << "Queueing" << endl;
    ArrayQueue queue;
    queue.Enqueue(1);
    queue.Enqueue(5);
    queue.peek();
```

```
    queue.Enqueue(4);
    queue.Dequeue();
    queue.Dequeue();
    queue.print();
    queue.Enqueue(5);
    queue.Enqueue(23);
    queue.Enqueue(0);
    queue.Enqueue(67);
    queue.print();

    cout << endl;
    cout << "////////////////////////////////" <<  endl;
    cout << endl;

// For linked deque
    cout << "Dequeing" << endl;
    ArrayDeques deque;
    deque.pushFront(1);
    deque.pushFront(5);
    deque.pushFront(4);
    deque.PopBack();
    deque.print();
    deque.pushBack(5);
    deque.pushBack(23);
    deque.pushFront(67);
    deque.print();

    cout << endl;
    cout << "////////////////////////////////" <<  endl;
    cout << endl;



    return 0;
}
```